Сопроводительная документация

Разряд — веб-платформа обратного подбора ИТ-специалистов. Работодатель описывает, кто нужен, получает объяснимую подборку людей категории «специализация + подтверждённый тестом грейд» и сам отправляет приглашение с зарплатной вилкой. Контакт кандидата открывается после принятия приглашения или собственного отклика.

Документ закрывает пункт 3.4 технического задания «ФСП спец. трек ТЗ»: архитектура, тест, подбор, валидация, реестр ФСП, API, версии библиотек и локальный запуск. Продуктовые решения, принятые до кода, лежат в specifikaciya.md. Этот файл описывает то, что реально собрано.

Дата прогона проверок: 11 октября 2026.

---

1. Что сделано по ТЗ

Обязательный контур хакатона работает: два кабинета, регистрация с согласиями, тест и категория, потребность, подборка с объяснением, приглашение с вилкой, скрытые контакты, пустая история ФСП, поиск по банку, вакансия и отклик.

Требование ТЗ	Статус
Кабинеты кандидата и работодателя	Сделано
Профиль вручную: ФИО, контакты, стек, роли, стаж, софт-скиллы, вилка	Сделано
Загрузка PDF-резюме с распознаванием и автогенерация PDF-профиля	Не сделано. Заполнение идёт формой. PDF — альтернатива в том же пункте ТЗ, на хакатон не брали
Опрос отрасли и специализации, выбор грейда, тест, категория	Сделано. Отрасль в MVP одна: информационные технологии
Грейд не понижается молча; попытка ниже или выше; смена не чаще раза в 90 дней	Сделано. Учебные почты `@example.com` кулдаун не ждут, чтобы можно было показать путь сразу
Приглашения: компания, текст, вилка, принять или отклонить, статусы у обеих сторон	Сделано
Контакты кандидата закрыты до принятия или собственного отклика	Сделано, покрыто тестом
Вилка «от–до» в рублях обязательна в приглашении и в вакансии	Сделано
Приватность, согласие на обработку и публикацию, привязка ID ФСП, пустая история	Сделано
Потребность, подборка, объяснение попадания	Сделано
Поиск по банку: специализация, грейд, стек, наличие достижений ФСП, без потери потребности	Сделано. В подборке вакансии фильтр «Только с достижениями ФСП» сужает список и не переписывает вакансию. Отдельная страница «Банк» ищет по всей базе
Регистрация по почте с подтверждением	Сделано в API. На учебном стенде `AUTO_VERIFY=1`: адрес считается подтверждённым сразу. Если выключить, токен печатается в лог API, подтверждение — `POST /api/v1/auth/verify-email`
Вакансии, редактирование, отклик и статусы	Сделано как дополнительный контур. Создание и правка идут через карточку потребности: вместе с ней публикуется вакансия. Кандидат

Требование ТЗ	Статус
	видит вакансии не выше своего грейда и статус своего отклика. Работодатель видит число откликов на карточке
Регулярные короткие задания от работодателя	Не сделано. В ТЗ это желательно, но не обязательно для MVP
Защита от фиктивных вакансий и стыковка с ATS	Концепция в разделе 7, рабочей модерации и коннектора нет. ТЗ выносит верификацию компании за MVP
OpenAPI, Docker, открытый код	Сделано. Лицензия MIT. Swagger: `/docs`
Презентация pptx/pdf и публичный URL стенда	В репозитории нет. Если стенда нет, ТЗ принимает видео и эту инструкцию запуска

Каталог `strings` в запуск не входит. Это отдельный набор экранов, не сервис «Разряд».

---

2. Архитектура

Один модульный монолит и одно веб-приложение на обе роли. Микросервисы при двух разработчиках и десятках пользователей добавили бы сеть отказов и ничего к сценарию жюри.

```
Браузер
  React + TypeScript (Vite)
        |  JSON, JWT, префикс /api/v1
        v
  FastAPI
    auth         регистрация, подтверждение почты, JWT, роли
    catalog      специализации, грейды, теги стека
    candidate    профиль, приватность, ФСП, тест, приглашения, отклики
    employer     компания, потребность, подборка, банк, приглашения, вакансии
    bank         карта навыков и открытый банк заданий
    matching     ворота категории, соответствие, текст объяснения
    assessment   сборка попытки, ключ, грейд, кулдаун
    validation   отчёт по золотым меткам
        |
        v
  PostgreSQL 16 в Docker, либо SQLite-файл при локальном запуске без контейнеров
```

В Docker интерфейс отдаёт nginx. Запросы `/api/` проксируются на API. Локально Vite проксирует `/api` на порт 8000.

Фоновой очереди нет. Пароли хранятся как bcrypt. Access-токен — JWT HS256 на 15 минут, refresh — случайная строка, в базе лежит только её SHA-256, срок 14 дней. Секрет подписи берётся из `JWT_SECRET` и в репозиторий не кладётся: в примере и в compose стоит учебное значение `dev-only-change-me`.

Роль проверяется на каждом закрытом маршруте. Чужое приглашение и чужая вакансия отвечают 404. Кандидат не читает чужие профили и чужие баллы.

Согласие на обработку персональных данных и согласие на публикацию профиля — два отдельных документа версии `2026-10-06`. Без обоих регистрация не проходит. Снятие публикации убирает человека из подборок и поиска.

В пользователе зарезервированы `external_issuer` и `external_subject` под будущий ФСП ID на Keycloak. Сейчас выпускается свой JWT. Таблицу пользователей для федеративного входа переписывать не нужно.

Ориентир производительности ТЗ — десятки одновременных пользователей. Подборка считается в запросе по базе демо-набора, без вызова модели. Генерация теста — выборка из банка в `data/`, тоже без модели на пути кандидата.

---

3. Механика тестирования

Название схемы: калиброванная сборка. Два человека одной специальности и одного заявленного грейда получают разные вопросы и одну и ту же форму сложности. Уникальное условие от нейросети в момент экзамена в оценку не входит: у него нет заранее согласованной сложности и ключа. Черновик задания модель может написать заранее. В экзамен попадает задание из банка, балл ставит ключ.

Почему это устойчиво к расхождению заданий

Угроза	Что сделано
Один вариант разойдётся по чатам	Состав собирается заново на каждую попытку. Порядок вариантов ответа перемешивается. Клиент не присылает состав теста и не получает ключ до отправки
Сложность несравнима	Форма фиксирована: 60 вопросов и 3 задачи. Смесь грейдов одинаковая
Оценка субъективна	Свободный текст грейд не двигает. Теория сверяется с ключом на сервере, код — со скрытыми тестами

Форма на 60 минут

Теория — 30 минут, 60 вопросов. Практика — 30 минут, 3 задачи на Python 3.12.

Смесь теории для заявленного грейда, у которого есть сосед снизу и сверху:

Откуда вопросы	Сколько	Из них софт-навыки
Свой грейд	42	4
Грейд ниже	12	1
Грейд выше	6	1

Около четверти вопросов — множественный выбор, остальные — одиночный. Навыки берутся из карты должностей `data/skill_map.json`: в первую очередь экзаменационные и уровни core / expected / common. Если в строгой выборке не хватает вопросов, добор идёт из того же грейда.

Практика — три роли: разминка (сложность на 1 ниже целевой), целевая, задача выше грейда. У каждой задачи 5 открытых тестов и 15 скрытых. Кандидат видит открытые и число скрытых. Эталонные входы скрытых тестов в браузер не уходят.

Балл:

```
балл = 100 × (0,5 × доля верной теории + 0,5 × доля верной практики)
```

Балл	Итог
ниже 65	Заявленный грейд не подтверждён. Категория сама не понижается
65–84	Грейд подтверждён и запечатывается
85 и выше на первом тесте	Грейд можно оставить или один раз попробовать уровень выше

После запечатывания смена грейда — только на соседний уровень и не чаще одного раза в 90 дней. Провал попытки смены оставляет текущую категорию. Провал Junior даёт явную полку «Стажёр», без отдельного банка. Повтор проваленного первого теста у обычного пользователя — через 7 дней. Lead тестом хакатона не назначается как повышение выше Senior по отдельному банку Lead: в шкале он есть, отдельного банка на Lead нет.

Раннер кода — отдельный процесс Python 3.12 с таймаутом 3 секунды и запретом `import`, файлов и служебных имён. Это учебная песочница, не изолированный продакшен-раннер. В контейнере API стоит Python 3.12, поэтому прогон задач там работает. Локальный интерпретатор 3.11 API поднимет, но задачи не засчитает, пока рядом нет 3.12.

Живой экзамен есть по Backend, Frontend и QA. Остальные специализации есть в справочнике, банка под них нет.

---

4. Категория и подбор

Категория — связка специализации и грейда. Стек в категорию не входит: Go и Java одного грейда — одна полка. Стек уточняет ранг уже внутри потребности.

Специализации справочника: Backend, Frontend, QA, данные, DevOps, мобильная разработка, ML, информационная безопасность. Грейды: Стажёр, Junior, Middle, Senior, Lead.

В подборку человек попадает только если профиль опубликован, показ разрешён, категория запечатана, специализация совпала с потребностью и грейд входит в запрошенный диапазон. Текст резюме ворота не открывает.

Сила профиля:

```
сила = 0,70 × балл теста + 0,30 × ФСП
```

Если у потребности включено «сначала участники ФСП» (`prefer_fsp` в API), веса становятся 0,55 и 0,45. В интерфейсе создания вакансии переключатель не выведен, по умолчанию он выключен.

Достижения ФСП. Лучший результат: участие без места — 30, топ-10% — 60, 3 место — 75, 2 место — 85, 1 место — 100. Множитель уровня: вузовское 0,4, региональное 0,7, национальное 1,0, международное 1,2. Итог обрезается на 100. Второй результат добавляет 15% своей величины, остальные не суммируются. Истории нет — ноль, это не штраф сверх нуля и не повод выкинуть человека.

Соответствие потребности, по нему сортируется список:

```
соответствие = (0,50 × сила / 100 + 0,35 × стек + 0,15 × вилка) × свежесть
```

Стек — доля обязательных навыков вакансии, которые кандидат отметил. Навыков нет — 1. Вилка — доля ожиданий кандидата, которая пересекается с вилкой вакансии. Ожидания не указаны — 1, это не бонус и не штраф. Вилки не пересекаются — 0. Свежесть: 1,00, если подтверждающему тесту меньше 90 дней, иначе 0,85.

На карточке соответствие округляется до целого процента. Порядок считается по точному значению. Рядом четыре полоски (тест, стек, ФСП, соответствие) и отметки «почему в топе»: категория, стек, тест, ФСП, условия. Если истории ФСП нет, на карточке написано: «Истории ФСП нет, ранг без этого бонуса».

Фильтр «Только с достижениями ФСП» убирает карточки без достижений и формулу не меняет. Снятие фильтра возвращает прежний состав той же вакансии.

---

5. Валидация

Золотые метки размечены в сидере и не вычисляются формулой соответствия. Метка пары «потребность — кандидат»: 2 — релевантен (специализация, грейд и обязательный стек совпали), 1 — категория совпала, стек совпал не весь, 0 — иная специализация или грейд вне диапазона.

Precision@5 — доля меток 2 среди первых карточек. Если людей меньше пяти, знаменатель — фактическая длина топа. Порог из спецификации: среднее ≥ 0,60. Доля меток 0 в топе: порог ≤ 0,10.

Прогон `GET /api/v1/admin/validation/summary` под `admin@example.com` на учебном сиде 11 октября 2026:

Потребность	Precision@5	Доля нерелевантных
Middle Backend на Go	1,000	0
Middle Frontend, React	0,750	0
Middle QA	0,667	0
Senior Backend на Go	0,333	0
Среднее	0,688	0,000

Порог среднего Precision@5 пройден. Нерелевантных (метка 0) в топах нет. Низкое значение у Senior Backend объясняется короткой полкой: в топе три человека, один релевантен, двое частично совпали по стеку. Это метка 1, не чужая категория.

Автотест `backend/tests/test_flow.py` дополнительно проверяет: контакты закрыты до принятия, вилка обязательна, Игорь без ФСП есть в подборке и уходит под фильтром, провал теста не назначает грейд, форма содержит 60+3 задания без ключа в ответе, вакансия выше грейда кандидату не видна, регистрация без обоих согласий отклоняется. Прогон 11 октября 2026: 9 тестов, все прошли.

Отдельные проверки из плана спецификации в этой сдаче не проводились, числа по ним не подставлялись: стабильность полки на пяти формах, разрыв сильного и слабого кандидата на форме Middle, согласие с экспертной меткой на 12 протоколах, пилот на пяти знакомых. Объяснение карточки собирается из тех же чисел, что и сортировка; автотест сверяет набор факторов и процент на карточке с точным значением.

---

6. Реестр ФСП

Боевого API на хакатоне нет. Контракт такой: кандидат вводит ID участника, сервер ищет достижения в таблице `fsp_achievements`. Неизвестный ID даёт ошибку и не создаёт пустых наград. Отвязка доступна в профиле. Источник демо-строк — `seed-registry`.

Поля, которые имеет смысл получать из реестра: идентификатор участника, дисциплина, соревнование, дата, уровень, место или процентиль, роль в команде, источник подтверждения. На карточке показываются до двух достижений, уровень и место. Полный список приходит в объекте профиля.

Нет истории — штатное состояние. Бейдж достижения не рисуется, в объяснении явная фраза про отсутствие бонуса, фильтр «только ФСП» такого человека скрывает и возвращает. Категория, тест и приглашение работают как у остальных.

Дальше тот же `participant_id` можно связать с `external_subject` Keycloak, не меняя формулу ранга.

---

7. Что сознательно осталось концепцией

Защита от фиктивных вакансий. Следующая фаза, не галочка в прототипе: уровни доверия компании (почта, корпоративный домен, ИНН), метка вилки вне ориентира грейда без запрета отправки, суточный лимит приглашений, жалоба кандидата, аудит текста после отправки. Верификацию компании ТЗ на MVP не требует.

ATS. «Разряд» остаётся местом согласия кандидата, ATS — местом этапов. Наружу ушли бы идемпотентные события: приглашение принято (контакт только в этом событии), приглашение отклонено. Ключ интеграции — отдельный от пользовательского JWT, с правом только на свою компанию. Рабочего коннектора нет.

---

8. API

Базовый префикс `/api/v1`. Спецификация OpenAPI собирается из кода: `/openapi.json`, Swagger UI — `/docs`. Ошибка:

```
{"error": {"code": "SALARY_REQUIRED", "message": "текст", "details": {}}}
```

Коды, которые встречаются в сценарии: 401 `UNAUTHORIZED`, 403 `FORBIDDEN` и `EMAIL_NOT_VERIFIED`, 404 `NOT_FOUND`, 409 `EMAIL_TAKEN`, `GRADE_COOLDOWN`, `INVITE_EXISTS`, 422 `VALIDATION`, `CONSENT_REQUIRED`, `SALARY_REQUIRED`, `BANK_EMPTY`, 429 `TOO_MANY`.

Закрытые методы ждут `Authorization: Bearer <access>`.

Метод	Путь	Кто
POST	`/auth/register`	гость
POST	`/auth/verify-email`	гость
POST	`/auth/login`	гость
POST	`/auth/refresh`	гость
POST	`/auth/logout`	сессия
GET	`/auth/session`	любой вошедший
GET	`/catalog/specializations`	любой вошедший
GET	`/catalog/grades`	любой вошедший
GET	`/catalog/stack-tags`	любой вошедший
GET	`/public/stats`	публично
GET, PATCH	`/me`	кандидат
PUT	`/me/privacy`	кандидат
POST, DELETE	`/me/fsp-link`	кандидат
POST	`/me/grade/confirm`, `/me/grade/intern`	кандидат
POST	`/assessments/sessions`	кандидат
GET	`/assessments/current`	кандидат
GET, PUT	`/assessments/sessions/{id}`	кандидат
POST	`/assessments/sessions/{id}/submit`, `.../run`	кандидат
POST	`/assessments/code/lint`, `/assessments/code/format`	кандидат
GET	`/me/invitations`	кандидат
POST	`/me/invitations/{id}/accept`, `.../decline`	кандидат
GET	`/vacancies`	кандидат
POST	`/vacancies/{id}/applications`	кандидат
GET	`/me/applications`	кандидат
GET, PATCH	`/company`	работодатель
GET, POST	`/needs`	работодатель

Метод	Путь	Кто
GET, PATCH	`/needs/{id}`	работодатель
GET, POST	`/needs/{id}/shortlist`	работодатель
GET	`/candidates/search`	работодатель
GET	`/candidates/{id}`	работодатель
POST	`/invitations`	работодатель
GET	`/invitations`	работодатель
POST	`/vacancies`	работодатель
GET	`/company/vacancies`	работодатель
GET	`/vacancies/{id}/applications`	работодатель
GET	`/bank/overview` и вложенные `/bank/...`	публично, чтение банка
GET	`/health`	публично
GET	`/admin/validation/summary`	admin

Фильтры подборки и банка передаются query-параметрами: `grade`, `stack` (коды через запятую), `fsp_only=true`. Поиск банка дополнительно принимает `specialization`.

---

9. Библиотеки и версии

Версии сняты с `backend/requirements.txt` и `frontend/package-lock.json`.

Компонент	Версия
Python в образе API	3.12
FastAPI	0.115.6
Uvicorn	0.34.0
SQLAlchemy	2.0.36
Pydantic	2.10.4
pydantic-settings	2.7.0
psycopg	3.2.3
PyJWT	2.10.1
bcrypt	4.2.1
email-validator	2.2.0
pytest	8.3.4
httpx	0.28.1
black	24.10.0
PostgreSQL	16
nginx	1.27
Node в образе интерфейса	22
React	19.3.0
React DOM	19.3.0
React Router	7.18.4
Vite	6.4.4
TypeScript	5.9.3
@uiw/react-codemirror	4.25.12

---

10. Локальный запуск для проверки

Нужен один из двух способов. Для полного теста с задачами на код удобнее Docker: внутри API уже есть Python 3.12. Порты `5432`, `8000` и `8080` должны быть свободны.

Docker

1. Установить Docker Desktop. На Windows 10/11 нужен бэкенд WSL 2. Проверка: `docker compose version`.
2. Склонировать репозиторий и открыть его корень.
3. Запустить:

```
docker compose up --build -d
```

Секрет, сид и автоподтверждение почты уже заданы в `docker-compose.yml` (`JWT_SECRET`, `SEED=1`, `AUTO_VERIFY=1`). Отдельный `.env` для compose не нужен.

4. Дождаться статуса `Up` у сервисов `db`, `api` и `web`: `docker compose ps`.
5. Открыть интерфейс: http://127.0.0.1:8080
6. OpenAPI: http://127.0.0.1:8000/docs
7. Проверка API: http://127.0.0.1:8000/api/v1/health — ответ `{"status":"ok"}`.

Остановка без удаления данных: `docker compose stop`. Снова поднять: `docker compose start`. Убрать контейнеры и оставить базу: `docker compose down`. Удалить и базу: `docker compose down -v`.

Логи: `docker compose logs -f api`. После правок кода: `docker compose up --build -d`.

Без Docker

Нужны Python 3.11+ и Node.js 20+. База — файл `backend/razryad.db`. Для зачёта практических задач рядом должен находиться Python 3.12 (переменная `PYTHON312` или команда `python3.12`).

```
cd backend
copy .env.example .env
python -m venv .venv
.\.venv\Scripts\python -m pip install -r requirements.txt
.\.venv\Scripts\python -m uvicorn app.main:app --reload --port 8000
```

Второе окно:

```
cd frontend
npm install
npm run dev
```

Интерфейс: http://127.0.0.1:5173. Запросы `/api` Vite отправляет на порт 8000.

Автотесты:

```
cd backend
.\.venv\Scripts\python -m pytest tests -q
```

Учебные входы

Пароль у всех: `demo1234`

Кто	Почта	Зачем
Анна Ковалева	anna@example.com	Кандидат с достижением ФСП, грейд ещё не запечатан
Игорь Смирнов	igor@example.com	Backend · Middle, истории ФСП нет
Марина	marina@example.com	Работодатель, компания «Северная линия»
Админ отчёта	admin@example.com	Только API отчёта валидации, кабинета в интерфейсе нет

Сценарий проверки

1. Войти Мариной. Открыть вакансию «Middle Backend на Go» (в интерфейсе это Backend-разработка, Middle).
2. В списке есть Игорь. На его карточке прямо сказано, что истории ФСП нет. Контакта нет.
3. Включить «Только с достижениями ФСП». Игорь исчезает, вакансия на месте. Выключить — Игорь возвращается.
4. Открыть «Банк». Поставить специализацию Backend и грейд Middle. Список сужается. Галочка ФСП снова убирает Игоря.
5. Вернуться в вакансию и пригласить человека с достижением ФСП. Вилка подставится из вакансии. Контакт на карточке ещё закрыт.
6. Выйти, войти этим кандидатом, открыть «Приглашения», увидеть компанию, текст и вилку, принять.
7. Снова войти Мариной, открыть «Отклики»: статус «принято», появились почта и telegram.
8. Под тем же кандидатом открыть «Вакансии» и откликнуться. Повторный отклик на ту же вакансию не создаётся. Вакансия выше своего грейда в списке не появляется. Анна без запечатанного грейда список вакансий не видит.
9. Живой тест: войти Анной, «Тест», специализация Backend, грейд Middle. 60 вопросов и 3 задачи, 60 минут. Ключ в браузер не отдаётся. Балл ниже 65 категорию не назначает. Балл от 65 запечатывает Backend · Middle.

Карта навыков и банк вопросов собираются скриптами в `data/` и смотрятся на странице «Навыки и задания» без входа в кабинет работодателя. Для проверки подбора их пересобирать не нужно: сид уже лежит в репозитории.